BRAHAM OF LONDON ENGINEERING MANU

SION 2.0 | APRIL 2026 | INTERNAL

Abraham of London — Engineering Manual

Version: 2.0
Date: April 2026
Classification: Internal — Engineering
Repository: `aol-check-visual`

PREAMBLE

Abraham of London is a Decision Authority Infrastructure platform. It provides diagnostic, alignment, and governance tooling for individuals and enterprises making consequential decisions. The system scores decision conditions, detects contradictions, produces governed synthesis, and tracks decision memory over time.

The platform is a commercial delivery mechanism built on Next.js 16, TypeScript strict mode, PostgreSQL (Neon), and deployed to Netlify.

QUICK START

```
git clone <repo-url> && cd aol-check-visual
pnpm install
npx prisma generate
pnpm dev
```

Key Commands

Command	Purpose
`pnpm dev`	Start development server (runs MDX gate + contentlayer build first)
`pnpm build`	Production build (generates EPUBs, then next build)
`pnpm quality:full`	Run `scripts/quality/full-validation.mjs` — the only acceptable clean bill
`pnpm test:unit`	Run Vitest unit + integration tests
`pnpm test:e2e`	Run Playwright end-to-end tests (separate runner)

SYSTEM DNA — DATA MODEL OVERVIEW

The platform is organized in five layers:

```
┌─────────────────────────────────────────────────────────────┐
│  CLIENT LAYER                                               │
│  Next.js 16.2 · React 19 · Tailwind · Pages Router (primary)  │
│  + App Router (admin/enterprise)                            │
├─────────────────────────────────────────────────────────────┤
│  APPLICATION LAYER                                          │
│  Server Components · API Routes · Server-Only Services      │
├─────────────────────────────────────────────────────────────┤
│  DOMAIN LAYER                                               │
│  Diagnostic Engine · Synthesis Engine (server-only)         │
│  Product Elevation Layer · Arbiter Tournament               │
├─────────────────────────────────────────────────────────────┤
│  DATA LAYER                                                 │
│  Prisma ORM · PostgreSQL (Neon) — sole authority            │
│  No SQLite in production. 126 models.                       │
├─────────────────────────────────────────────────────────────┤
│  INFRASTRUCTURE                                             │
│  Netlify · Neon · Upstash (optional cache) · Resend · Stripe  │
└─────────────────────────────────────────────────────────────┘
```

PART I — SYSTEM ARCHITECTURE

Chapter 1: Platform Overview

Abraham of London is a full-stack Next.js 16 application written in TypeScript strict mode. It serves as a commercial delivery mechanism for decision authority diagnostics, alignment campaigns, strategy room sessions, and enterprise governance reporting.

The platform operates across two routing systems. The Pages Router (`pages/`) handles the primary user-facing experience: diagnostics, strategy room, content delivery, authentication, and the commercial surface. The App Router (`app/`) handles admin dashboards, enterprise campaign management, PDF rendering, purpose alignment, and the strategy room execution interface.

All scoring, classification, and synthesis logic runs server-side only. The client collects user input, submits it to API routes, and renders the public DTO that comes back. No thresholds, weights, classification rules, or scoring formulas exist in the client bundle.

The system enforces intellectual property protection through a layered architecture: public pages use controlled ambiguity ("proprietary scoring model," "multi-factor evaluation"), the API returns only sanitized DTOs, and the scoring engine imports `server-only` to prevent accidental client bundling.

Every diagnostic produces an IntelligenceSpine — a single contract object that flows through every stage from fast diagnostic through constitutional assessment, team validation, enterprise pricing, executive reporting, strategy room simulation, and outcome verification. No stage computes in isolation. No stage re-derives context from zero.

Chapter 2: Technology Stack

Runtime

- **Next.js:** 16.2.2
- **React:** 19
- **TypeScript:** 5.9.3, strict mode with `noUncheckedIndexedAccess` , `noImplicitOverride` , `noFallthroughCasesInSwitch`
- **Node.js:** >=20.0.0
- **Package Manager:** pnpm >=9.0.0

Database

PostgreSQL via Neon is the authoritative data store for ALL persistent data. The Prisma schema (`prisma/schema.prisma`) is the single source of truth with 126 models. There is no SQLite in production. Legacy `db:sqlite:*` scripts exist in `package.json` but are not part of the production data path.

Cache

Redis via Upstash is OPTIONAL. The system functions without it. When Redis is unavailable, rate limiting falls back to PostgreSQL (the `RateLimitBucket` model). If both are unavailable on critical routes, the system fails closed (request denied).

ORM

Prisma with `schema.prisma` as the single source of truth. The generated client outputs to `node_modules/.prisma/client`. Migrations use `prisma migrate dev` for development and `prisma migrate deploy` for production.

Deployment

Netlify. Not Vercel. The `netlify.toml` configures the build command (`pnpm build:netlify`), Node 20.20.0, and the `@netlify/plugin-nextjs` adapter. The publish directory is `.next`.

Email

Resend for transactional email delivery.

Payments

Stripe for checkout, entitlement verification, and report purchases.

Security

- **Input validation:** zod schemas on all API routes
- **Sanitization:** DOMPurify for user-generated content
- **Encryption:** AES-256-GCM for spine data in sessionStorage (per-session key via Web Crypto API)
- **Server-only scoring:** `import "server-only"` on all scoring and synthesis modules
- **Rate limiting:** Redis primary, Postgres fallback, fail-closed on critical routes

Chapter 3: Application Architecture

Routing

Pages Router (`pages/`) is the primary router. It handles:

- Fast diagnostic flow and submission
- Constitutional intake and reports
- Strategy room intake
- Content pages (briefs, canon, books, blog, artifacts)
- Authentication (`pages/api/auth/[...nextauth].ts`)
- All diagnostic API routes (`pages/api/diagnostics/`)
- Commercial endpoints (Stripe checkout, downloads)

App Router (`app/`) handles:

- Admin dashboard (`app/admin/`)
- Enterprise campaign management (`app/enterprise/`)
- PDF rendering (`app/__pdf/` , `app/render/`)
- Purpose alignment assessment (`app/purpose-alignment/`)
- Strategy room execution sessions (`app/strategy-room/`)
- Downloads management (`app/downloads/`)
- Registry and settings

Route Groups

The App Router uses route groups for layout isolation:

- `(dashboard)` — authenticated dashboard shell
- `admin` — admin-only routes
- `enterprise` — organisation-scoped routes

Rendering Strategies

- **Static Generation:** Content pages (briefs, canon, books) via Contentlayer2
- **Server-Side Rendering:** Diagnostic results, authenticated pages
- **Client Components:** Interactive diagnostic forms, conversion surfaces
- **Server Components:** Admin views, report rendering, data-heavy pages

---

PART II — CORE SYSTEMS

Chapter 4: Database Layer

PostgreSQL (Neon) is the sole persistent data authority. The Prisma schema contains 126 models organized into the following domains:

Identity & Access

`User` , `Account` , `VerificationToken` , `Entitlement` , `AccessKey` , `AccessKeyUse` , `AccessAuditLog` , `AccessInvite` , `Session` , `AdminSession` , `ApiKey` , `ApiLog` , `InnerCircleKey` , `MfaSetup` , `SecurityLog`

Organisation & Campaigns

`Organisation` , `AlignmentCampaign` , `AlignmentSnapshot` , `OrganisationInvite` , `OrganisationMembership` , `CampaignParticipant` , `EnterpriseAssessment` , `TeamAssessmentSnapshot` , `TeamAssessmentCampaign` , `TeamAssessmentInvite` , `TeamAssessmentResponse` , `TeamAssessmentAggregate` , `OrganisationAssessmentSnapshot` , `LeadershipGapSnapshot` , `EnterpriseReport` , `ExecutiveReportingRun` , `ExecutiveReportingArtifact`

Diagnostics

`DiagnosticRecord` , `DiagnosticReportOrder` , `DiagnosticArtifact` , `DiagnosticRegenerationJob` , `DiagnosticAuditEvent` , `DiagnosticArtifactAccessGrant` , `DiagnosticLineageEvent` , `DiagnosticJourney` , `DiagnosticEvidenceNode` , `DiagnosticDecisionObject` , `DiagnosticStageRecord` , `DiagnosticThreadSnapshot` , `ConstitutionalIntakeReport`

Decision Infrastructure

`DecisionMemory` , `DecisionDependency` , `DecisionStakeholder` , `StakeholderPosition` , `DecisionRecommendationSession` , `DecisionRecommendationImpression` , `DecisionRecommendationClick` , `DecisionRecommendationConversion` , `DecisionSessionFollowup` , `DecisionAssetEfficacy` , `DecisionAssetPerformance` , `DecisionAssetContextPerformance` , `DecisionAssetGovernanceRule` , `DecisionSignalRegistry` , `DecisionGovernanceAlert` , `DecisionJourneyEvent` , `ConsequenceTimeline` , `EscalationEvent` , `CalibrationState` , `CalibrationEvent` , `PatternBreakerContract`

Strategy & Governance

`StrategyRoomSession` , `StrategyRoomRecommendationImpression` , `StrategyRoomFollowup` , `StrategyRoomConversion` , `StrategyRoomExecutionSession` , `StrategyDecisionLog` , `StrategyInquiry` , `StrategyIntake` , `StrategicIntervention` , `CorrectionNode` , `EnforcementPlaybook` , `PlaybookApplication` , `EnforcementCycle` , `RetainerContract` , `RetainedDecision`

Operations & Monitoring

`RateLimitBucket` , `RateLimitLog` , `GovernanceLog` , `GovernanceMetricDefinition` , `OperationalIncident` , `ArtifactManifest` , `JobDeadLetter` , `ServiceLevelSnapshot` , `RunbookEntry` , `MonitoringSnapshot` , `BenchmarkFact` , `BenchmarkCohortSnapshot` , `SystemAuditLog` , `AuditEvent` , `AuditResponse` , `FoundationTelemetryEvent`

Content & Commerce

`ContentMetadata` , `Framework` , `StrategicLink` , `ContentRelation` , `PrivateAnnotation` , `CanonEntry` , `StrategicFramework` , `DownloadAuditEvent` , `PrintAsset` , `PremiumDownloadToken` , `PremiumDownloadAttempt` , `FrameworkAccessLog` , `PageView` , `UserPreference` , `UserIntegration` , `InnerCircleMember` , `BillingCustomer` , `ClientEntitlement` , `ProofEvidence` , `FailedEntitlementGrant`

Purpose Alignment

`PurposeAlignmentAssessment` , `PurposeAlignmentReport` , `PurposeAlignmentReminderPreference` , `PurposeAlignmentReminderLog`

Deal Flow

`DealFlowSubmission`

Migration Commands

```
pnpm db:generate    # prisma generate
pnpm db:push        # prisma db push (schema sync without migration)
pnpm db:migrate     # prisma migrate dev (create migration)
pnpm db:deploy      # prisma migrate deploy (apply in production)
pnpm db:seed        # tsx prisma/seed.ts
pnpm db:studio      # prisma studio (browser GUI)
```

Chapter 5: Authentication & Authorization

Identity Authority

NextAuth with JWT is the PRIMARY identity authority. The configuration lives in `pages/api/auth/[...nextauth].ts` . JWT tokens carry user ID, email, and role. The `NEXTAUTH_SECRET` environment variable is required.

Cookie Architecture

- **`aol_access`** : Session presence indicator ONLY. It does NOT upgrade the user's tier. Its existence means "a session was established" but the tier must be resolved server-side via Prisma.
- **`next-auth.session-token`** / **`__Secure-next-auth.session-token`** : The actual NextAuth JWT. This is the sole identity credential.

Identity Resolution

`lib/server/auth/tokenStore.postgres.ts` exports `getSessionContext()` and `verifySession()`. These functions:

1. Decode the NextAuth JWT via `getToken()` from `next-auth/jwt`
2. Look up the user in Prisma via `getUserAccess()`
3. Return the resolved tier, role, and session metadata

The cookie alone does not determine tier. The server always resolves tier from the database.

Tier Hierarchy

```
public < member < inner_circle < client < architect < owner
```

Additional tiers exist in the `AccessTier` enum (`restricted` , `legacy` , `top_secret`) for specific access control scenarios.

Dev-Login

The `pages/api/auth/sovereign-login.ts` route exists for development convenience. It must return 404 in production. No development authentication bypass is permitted in deployed environments.

Rate Limiting

Priority chain:

1. **Redis (Upstash)** — primary store via `getRedis()`
2. **PostgreSQL** — fallback via `RateLimitBucket` model and `rate-limit-store.postgres.ts`
3. **Fail-closed** — if both are unavailable on critical routes, the request is denied

There is no in-memory rate limiting in production. The edge proxy uses a per-isolate `Map` for development convenience only — this does not persist across cold starts and is not suitable for production.

Rate limit configurations are defined in `lib/server/rate-limit-unified.ts` :

Key	Limit	Window
PUBLIC	120	60s
AUTH	60	60s
ADMIN	100	60s
API_STRICT	30	60s
API_GENERAL	100	1hr
INNER_CIRCLE_UNLOCK	30	10min
CONTACT	5	1hr
DOWNLOAD	20	1hr

Chapter 6: Content System

Contentlayer2

Content is managed via Contentlayer2, configured in `contentlayer.config.ts`. The system processes MDX files from the `content/` directory and generates typed document definitions in `.contentlayer/generated/`.

Document types are defined for briefs, dossiers, canon entries, lexicon entries, frameworks, books, editorials, and other content categories. Each type specifies required frontmatter fields, computed fields (slugs, routes), and validation rules.

MDX Processing

MDX files use remark-gfm for GitHub Flavored Markdown and rehype-slug + rehype-autolink-headings for navigation. The build pipeline includes:

- **MDX integrity check** (`scripts/mdx-integrity-check.mjs`): Validates that upstream scripts have not stripped or escaped MDX component tags
- **MDX gate** (`scripts/mdx-illegal-jsx-gate.mjs`): Blocks builds if illegal JSX patterns are detected

Both checks run as pre-commit hooks and during the build pipeline.

Canon Codes

Canon entries use a structured code system for referencing decision authority principles. The `CanonEntry` Prisma model stores canonical references with their codes, titles, and relationships.

Glossary

The glossary system (`scripts/glossary-injector.mjs`) injects term definitions into content at build time, ensuring consistent terminology across all published material.

Toolkits

Published toolkits contain conceptual models only. No numeric frameworks, scoring brackets, or axis names appear in public-facing toolkit content. This is an IP protection requirement.

PART III — DOMAIN SYSTEMS

Chapter 7: Diagnostic Engine

The diagnostic engine is the core domain system. It processes user-submitted decision situations through a multi-stage pipeline.

CaseObject

Defined in `lib/decision/case-object.ts`. The CaseObject captures 6 user inputs preserved verbatim:

1. `decision` — the decision in the user's own words
2. `priorAttempt` — what they already tried
3. `costOfDelay` — what gets more expensive each week
4. `claimedOwner` — who they say owns the decision
5. `blocker` — what they say is blocking the decision
6. `forcedAction` — what they would do if forced to decide in 24 hours

The CaseObject also carries derived fields: `contradiction`, `inferredAvoidance`, `conditionClass`, `signalStrength`, and `specificityScore`. These are computed server-side, never client-supplied.

C3 Fidelity Scorer

Defined in `lib/decision/c3-fidelity-scorer.ts`. Scores three dimensions:

- **Clarity** (0-1): Is the decision itself clear?
- **Context** (0-1): Is there enough surrounding information?
- **Consequence** (0-1): Is the cost/stakes articulated?

Tiered enforcement based on the composite specificity score:

Tier	Score Range	Behavior
HARD_RECOVERY	< 0.5	No synthesis. Recovery questions only.
SOFT_RECOVERY	0.5 - 0.7	Deterministic output only. No contradiction block.
FULL_SYNTHESIS	>= 0.7	Full LLM synthesis + arbiter tournament.

Synthesis Engine

Defined in `lib/decision/synthesis-engine.ts`. Imports `server-only` — cannot be bundled into the client.

The engine produces a `GovernedSynthesis` output containing: verdict, primary contradiction, avoided decision, why prior attempts failed, concrete move, default path forecast, signal strength, certainty boundary, and quoted user language.

Architecture:

1. CaseObject -> C3 Score -> tier check
2. HARD_RECOVERY -> recovery questions only
3. SOFT_RECOVERY -> deterministic fallback only
4. FULL_SYNTHESIS -> LLM synthesis -> arbiter tournament -> governed output
5. Arbiter rejection -> explicit mismatch message, NOT silent fallback

Phrasing variant rotation uses a hash-based approach to ensure deterministic but varied output across different cases.

Arbiter Tournament

Defined in `lib/decision/arbiter-tournament.ts`. Five mandatory rules — if any hard rule fails, synthesis is rejected:

1. **Condition integrity** — synthesis condition class must match deterministic classification
2. **Contradiction alignment** — must reference terms from the deterministic contradiction set
3. **Move validity** — must reference the stated blocker, not generic advice
4. **Cost consistency** — no invented costs without user-stated cost data
5. **Avoidance proof** — must reference forcedAction or priorAttempt mismatch

Violations are classified as `hard` (reject) or `soft` (warn but allow). The user sees the mismatch when hard violations occur. This transparency is intentional.

Severity

The scoring system in `lib/diagnostics/scoring.ts` uses 4 severity tiers:

Severity	Score Range
critical	< 40
high	40 - 59
moderate	60 - 79
low	>= 80

TARGET — not yet implemented: The full severity model targets 6 tiers (NEGLIGIBLE, LOW, MODERATE, HIGH, CRITICAL, SYSTEMIC). NEGLIGIBLE and SYSTEMIC are not yet present in `scoring.ts`.

Condition Classes

Four condition classes, defined in `lib/decision/case-object.ts`:

- `authority` — ownership and mandate failures
- `definition` — the decision itself is unclear
- `execution` — the decision is clear but execution stalls

- `instability` — structural instability across multiple dimensions

Contradiction Archetypes

Currently implemented in `lib/diagnostics/contradictions.ts` — 3 core archetypes with variant expressions:

1. **URGENCY_VS_OWNERSHIP** — high urgency + unclear ownership
2. **CLARITY_VS_ACCOUNTABILITY** — defined decision + no accountability
3. **URGENCY_VS_STATE** — urgent + repeatedly deferred

TARGET — not yet implemented: The full model targets 13 contradiction archetypes. The current 3 serve as the core set with additional archetypes planned.

Server-Side Scoring

ALL scoring runs server-side via `/api/diagnostics/score`. The client is submit-only:

1. Collects answers from the user
2. POSTs to the scoring API
3. Renders the public DTO that comes back

No thresholds, weights, formulas, or classification rules exist in the client bundle.

Chapter 8: Alignment & Campaign System

Purpose Alignment

The Purpose Alignment assessment (`app/purpose-alignment/`) evaluates individual alignment across six domains defined in the `AlignmentDomain` enum:

- IDENTITY
- DECISION
- ENVIRONMENT
- BEHAVIOUR
- EMOTIONAL_ORDER
- LEGACY

Results are classified into four bands (`AlignmentBand`): ALIGNED, DRIFTING, MISALIGNED, DISORDERED.

The `PurposeAlignmentAssessment` and `PurposeAlignmentReport` Prisma models store assessment data and generated reports. Reminder preferences and logs are tracked separately.

Enterprise Campaigns

Enterprise campaigns (`AlignmentCampaign` model) support organisation-wide diagnostic deployment. A campaign has:

- An organisation owner
- A lifecycle (draft -> intake -> active -> closed -> archived)

- Participants invited via `CampaignParticipant` with token-based access
- Team and organisation-level assessment snapshots
- Executive reporting runs that produce governance artifacts

The campaign system generates `OrganisationAssessmentSnapshot`, `TeamAssessmentSnapshot`, and `LeadershipGapSnapshot` records for comparative analysis.

OGR (Organisation Governance Report)

Executive reporting runs (`ExecutiveReportingRun`) produce artifacts (`ExecutiveReportingArtifact`) that summarize campaign findings at the board level. Reports are generated as PDFs via the App Router's PDF rendering pipeline.

Chapter 9: Strategy Room & Execution

Strategy Room

The Strategy Room (`app/strategy-room/`, `StrategyRoomSession` model) provides execution infrastructure for decisions that have been diagnosed. It operates through:

1. **Intake:** `StrategyIntake` and `StrategyInquiry` models capture the decision context
2. **Session management:** `StrategyRoomSession` tracks active execution sessions
3. **Execution tracking:** `StrategyRoomExecutionSession` and `StrategyDecisionLog` record actions taken
4. **Recommendation engine:** `StrategyRoomRecommendationImpression`, `StrategyRoomFollowup`, `StrategyRoomConversion` track recommendation effectiveness

Enforcement

The enforcement system uses:

- **Breach ladder:** Escalating consequences tracked via `EscalationEvent`
- **Enforcement playbooks:** `EnforcementPlaybook` and `PlaybookApplication` models
- **Enforcement cycles:** `EnforcementCycle` model for recurring governance enforcement
- **Pattern breaker contracts:** `PatternBreakerContract` model for formalizing commitment to change

Execution Tracking

- `DecisionJourneyEvent` records decision lifecycle events
- `ConsequenceTimeline` tracks projected and actual consequences
- `CalibrationState` and `CalibrationEvent` track calibration of the scoring system against real outcomes
- `OutcomeVerificationRecord` stores verified decision outcomes for longitudinal analysis

Chapter 10: Commercial Layer

Stripe Integration

Stripe handles payment processing for diagnostic reports and premium access. Key flows:

- **Checkout:** `pages/api/diagnostics/create-report-checkout.ts` creates Stripe checkout sessions for diagnostic report purchases
- **Webhooks:** `pages/api/webhooks/` processes Stripe events (payment success, subscription changes)
- **Entitlements:** `Entitlement` model grants access after successful payment. `ClientEntitlement` provides client-tier access.

Entitlement Verification

The `Entitlement` model supports three types (`EntitlementType`): TIER, PRODUCT, ARTIFACT. Each entitlement has a status (ACTIVE, REVOKED, EXPIRED) with start/expiry dates and audit trail (issuedBy, revokedBy, reason).

Report Purchase Flow

1. User completes diagnostic -> receives `diagnosticId` and `diagnosticRef`
2. User initiates report purchase -> system creates `DiagnosticReportOrder` + Stripe checkout
3. Payment confirmed -> `reportStatus` updated on `DiagnosticRecord`
4. Report generated -> `DiagnosticArtifact` created with PDF storage reference
5. User downloads via access-controlled route

PART IV — PRODUCT ELEVATION LAYER

Chapter 11: Cost of Inaction Engine

File: `lib/server/decision/cost-of-inaction.server.ts`

Server-only module (`import "server-only"`). Produces qualitative exposure assessments, not numeric calculations.

Input

```
type CostOfInactionInput = {
  state: "ORDERED" | "DRIFTING" | "MISALIGNED" | "DISORDERED";
  estimatedExposureGBP?: number | null;
  decisionWindow?: string | null;
  headcountAffected?: number | null;
  marketExposure?: string | null;
};
```

Output

```
type CostOfInactionPublic = {
  exposureBand: "low" | "moderate" | "high" | "critical" | "undisclosed";
  horizon30: string;
  horizon60: string;
  horizon90: string;
  executiveWarning: string;
};
```

The engine classifies exposure into bands based on state and financial anchors when available. It produces 30/60/90-day horizon narratives describing projected deterioration in natural language. No formulas or multipliers are exposed in the public output.

When financial data is not provided, the engine derives the band from state alone (DISORDERED -> critical, MISALIGNED -> high, DRIFTING -> moderate). The `undisclosed` band is used when no state or financial anchor is sufficient for classification.

Chapter 12: Execution Failure Predictor

File: `lib/server/decision/execution-failure.server.ts`

Server-only module. Performs state-based assessment of likely execution failure modes.

Input

```
type ExecutionFailureInput = {
  state: string;
  publicConditions?: string[];
  directive: string;
};
```

Output

```
type ExecutionFailurePublic = {
  likelyFailureMode: string;
  whyExecutionWillStall: string;
  requiredCorrection: string;
};
```

The engine pattern-matches on public-safe state and conditions to predict the specific failure mode. For example:

- DISORDERED -> "Authority collapse" — no single authority can enforce the directive
- MISALIGNED + authority conditions -> ownership ambiguity failures
- DRIFTING -> passive consensus seeking that produces no binding outcome

Each prediction includes why execution will stall and the required correction to unblock it.

Chapter 13: Decision Authority Index

File: `lib/server/decision/authority-index.server.ts`

Server-only module. Produces an interpretive governance band, not a numeric score.

Input

```
type DecisionAuthorityIndexInput = {
  state: string;
  escalationRequired?: boolean;
  repeatedConditions?: string[];
  escalationTrend?: string;
};
```

Output

```
type DecisionAuthorityIndexPublic = {
  band: "strong" | "strained" | "weak" | "critical";
  label: string;
  boardMeaning: string;
  nextGovernanceMove: string;
};
```

The index classifies decision authority into four bands:

- **critical** — decision authority has failed; board-level intervention indicated
- **weak** — governance structure is producing decisions but cannot enforce them
- **strained** — functional but showing signs of degradation
- **strong** — governance structure is producing and enforcing decisions

Each band carries a board-level meaning statement and a specific next governance move.

Chapter 14: Decision Memory Service

File: `lib/server/decision-memory/memory-service.server.ts`

Server-only module. CRUD operations plus trend analysis backed by the `DecisionMemory` Prisma model.

Operations

- `createDecisionMemory(input)` — persists a new decision memory record
- `listDecisionMemoryByUser(userId)` — retrieves up to 50 most recent records
- `summariseDecisionMemoryTrend(userId)` — computes trend analysis

Trend Analysis

```
type DecisionMemoryTrend = {
  totalDecisions: number;
  dominantState: string;
  repeatedConditions: string[];
  escalationTrend: "stable" | "rising" | "falling" | "insufficient_data";
  executiveSummary: string;
};
```

The trend analysis identifies the dominant decision state over time, detects repeated conditions, and computes escalation trend direction. Only public-safe summaries are returned — no internal scoring data, raw signals, or engine state appears in the trend output.

The `DecisionMemory` model stores: userId, organisationId, sessionId, source, state, headline, summary, directive, recommendations, publicSignals, escalationLabel, and escalationLevel.

PART V — PRESENTATION LAYER

Chapter 15: Styling

Design System: Institutional Monumentalism

The visual language is dark, authoritative, and precise. It communicates institutional weight, not startup energy.

Design Tokens

Token	Value
Gold (primary accent)	`#C9A96E` / `rgb(201 169 110)`
Gold Strong	`rgb(212 175 55)`
Surface (background)	`rgb(14 14 18)`
Surface 2	`rgb(9 9 12)`
Surface 3	`rgb(6 6 9)`

Typography

Role	Font
Serif (headings, institutional)	Cormorant Garamond
Sans (body, UI)	Inter (via `var(--font-sans)`)
Mono (code, data)	JetBrains Mono

Tailwind Configuration

Configured in `tailwind.config.cjs`. Dark mode uses the `class` strategy. The theme extends the default with:

- Custom color scales (gold, surface, ds.* design system tokens)
- Gold gradient backgrounds (`gold-radial`, `gold-linear`)
- Panel shadows with gold accents (`aol-panel-gold`)
- Custom animations (`pulse-gold`, `fade-in`)
- Container with centered layout, max-width 1440px

Content scanning covers `pages/`, `components/`, `app/`, `layouts/`, `lib/`, `content/`, `src/`, and `.contentlayer/generated/`.

Chapter 16: Component Architecture

Layout

The root layout wraps all pages with consistent navigation, footer, and meta configuration. The `_app.tsx` (Pages Router) and `app/layout.tsx` (App Router) provide their respective shells.

Unified Conversion Surface

The diagnostic result page assembles a conversion surface from modular components:

- **CaseActiveBanner** — displays the user's case reference and condition classification
- **ConsequenceTimeline** — visualizes the 7/30/90-day forecast with progressive deterioration
- **LimitationsBlock** — states what the system cannot conclude (certainty boundary)
- **DirectiveCTA** — the primary call to action based on the diagnostic finding
- **FeedbackLoop** — captures user response to the diagnostic output

ExecutiveDecisionAuthorityBlock

Renders the Decision Authority Index output. Shows the governance band, board meaning, and next governance move. Styled with institutional weight — gold borders, serif headings, dark surfaces.

Component Principles

- Components receive public DTOs only, never raw scoring data
- No component computes scores, classifications, or thresholds
- Interactive state uses React hooks (useState, useEffect, useReducer)
- No global state management library (no Redux, no Zustand)

Chapter 17: State Management

Client State

React built-in state only. Components use `useState` and `useEffect` for local state. Form state uses controlled components with React state.

Session Persistence

`sessionStorage` stores the public DTO after diagnostic completion. For the intelligence spine (which contains server-side data), the stored version is encrypted with AES-256-GCM using a per-session key generated via the Web Crypto API. This prevents inspection of spine data through browser dev tools.

No Global Store

There is no Redux, Zustand, Jotai, or other global state management library. State flows through:

1. API response -> component props (server-rendered pages)
2. API response -> React state (client-side fetches)
3. sessionStorage -> React state (page revisits within session)

This simplicity is intentional. The system's complexity lives in the server-side domain layer, not the client state graph.

PART VI — API & SECURITY LAYER

Chapter 18: API Architecture

All API routes live under `pages/api/`. Every route validates input with zod and applies rate limiting.

Core Diagnostic Routes

`POST /api/diagnostics/score`

Server-side Fast Diagnostic scoring. Accepts `{ answers: Record<string, string>, committed: boolean }`. Runs the full scoring pipeline (CaseObject -> C3 -> Synthesis -> Arbiter -> Elevation Layer). Returns `FastDiagnosticResult` DTO only.

Imports: `createCaseObject`, `classifyCondition`, `inferContradiction`, `scoreC3`, `synthesise`, `buildDeterministicOutput`, `forecastDefaultPath`, `createSpine`, `persistSpineToJourney`, `computeCostOfInaction`, `assessExecutionFailure`, `computeAuthorityIndex`, `createDecisionMemory`, `summariseDecisionMemoryTrend`.

`POST /api/diagnostics/submit`

Diagnostic submission endpoint. Validates and persists the diagnostic submission. Forwards to CRM. Returns `diagnosticId` and `diagnosticRef` ONLY. No score, severity, or verdict in the response.

`GET /api/diagnostics/constitutional-intake/report`

Returns `PublicConstitutionalResult` only. No raw bundle, scoring data, or engine internals.

Supporting Routes

- `/api/auth/[...nextauth]` — NextAuth authentication
- `/api/auth/identity` — identity resolution
- `/api/auth/session` — session status
- `/api/stripe/` — Stripe checkout and webhook handling
- `/api/diagnostics/create-report-checkout` — report purchase flow
- `/api/rate-limit/` — rate limit status endpoints
- `/api/health` — system health check
- `/api/cron/` — scheduled task endpoints

Chapter 19: Public DTO Contract

FastDiagnosticResult

```
type FastDiagnosticResult = {
  caseRef: string;
  condition: string;
  conditionLabel: string;
  signalStrength: "low" | "moderate" | "high";
  fullAnalysis: boolean;
  recoveryQuestion: string | null;
  synthesis: {
    verdict: string;
    primaryContradiction: string;
    avoidedDecision: string;
    whyPriorAttemptsFailed: string;
    concreteMove: string;
    defaultPathForecast: string;
    certaintyBoundary: string;
    quotedUserLanguage: string[];
  } | null;
  forecast: {
    alreadyIncurred?: string;
    sevenDays: string;
    thirtyDays: string;
    ninetyDays: string;
    optionCompression?: string;
    consequenceShift?: string;
    controlShiftSummary: string;
  } | null;
  contradictionText: string | null;
  arbiterMessage: string | null;
  stateToken: string;
  costOfInaction?: CostOfInactionPublic;
  executionFailure?: ExecutionFailurePublic;
  authorityIndex?: DecisionAuthorityIndexPublic;
  memoryTrend?: DecisionMemoryTrend;
};
```

PublicConstitutionalResult

```
type PublicConstitutionalResult = {
  state: "ORDERED" | "DRIFTING" | "MISALIGNED" | "DISORDERED";
  headline: string;
  summary: string;
  directive: string;
  recommendations: string[];
  escalation?: { required: boolean; label: string };
};
```

FORBIDDEN in Any API Response

The following fields must NEVER appear in any response body:

- `rawScore`
- `severityScore`

- `threshold`
- `weight`
- `matchedSignals`
- `matchedKeywords`
- `arbiterTrace`
- `engineMode`
- `classificationRules`

Chapter 20: IP Protection Architecture

Public Pages

No thresholds, weights, scoring brackets, or axis names appear on any public-facing page. The system uses controlled ambiguity:

- "Proprietary scoring model"
- "Multi-factor evaluation"
- "Dynamic thresholds"

These phrases replace specific technical descriptions in all user-facing content.

Toolkits

Published toolkits contain conceptual models only. No numeric frameworks, exact scoring ranges, or weighted factor lists appear in downloadable materials.

Enforcement Signal Evidence

When the system presents evidence of enforcement signals, it uses abstracted descriptions. No exact threshold values, weight coefficients, or signal identifiers appear in the output.

DeterminismProof

The determinism proof display shows "signal strength" and "verification passed" only. No internal scoring data, arbiter trace, or classification rules are surfaced.

SessionStorage Encryption

The intelligence spine stored in sessionStorage is encrypted with AES-256-GCM. The encryption key is generated per-session via the Web Crypto API and held only in memory. This prevents casual inspection of spine data through browser developer tools. The key does not persist across page reloads — the spine must be re-fetched from the server.

Chapter 21: Rate Limiting

Architecture

Rate limiting uses a three-tier fallback chain:

1. **Redis (Upstash)** — primary store. Uses atomic increment with TTL via `getRedis()`.
2. **PostgreSQL** — fallback. Uses the `RateLimitBucket` model with `routeKey`, `identityKey`, `count`, and `windowStart` fields. Implemented in `lib/server/security/rate-limit-store.postgres.ts`.
3. **Fail-closed** — if both stores are unavailable, critical routes deny the request. The `unavailableResult()` function returns `allowed: false`.

Configuration

All rate limit configurations are centralized in `lib/server/rate-limit-unified.ts`. Each configuration specifies `limit`, `windowMs`, and `keyPrefix`.

Edge Proxy

The edge middleware uses a per-isolate `Map` for rate limiting. This is a development convenience only. It does not persist across cold starts, does not share state between isolates, and is not suitable for production rate limiting.

No In-Memory Production Rate Limiting

There is no in-memory rate limiting in production. All production rate limiting flows through Redis or PostgreSQL.

Chapter 22: Auth Security

Identity Authority

NextAuth/JWT is the sole identity authority. The `getSessionContext()` function in `lib/server/auth/tokenStore.postgres.ts` is the canonical way to resolve a user's identity and access tier.

Cookie Security

The `aol_access` cookie is a session presence indicator only. It does NOT upgrade the user's tier. The server always resolves tier from the database via Prisma.

Edge Behavior

For edge middleware where Prisma is not available, cookie-only requests receive a minimum "member" tier. This allows basic access control at the edge without database access, but does not grant elevated privileges.

Dev-Login Security

The `pages/api/auth/sovereign-login.ts` route is for development only. It must return 404 in production environments. This is enforced by checking `NODE_ENV`.

Integrity Scoring

The diagnostic system includes integrity detection that identifies:

- **Intent flips** — contradictions between stated intent and actual behavior
- **Cost swings** — dramatic changes in stated cost/urgency between assessments
- **Repeated breaches** — recurring pattern violations tracked via `PatternBreakerContract`
- **False authority** — claims of ownership that contradict other inputs

These integrity signals feed into the arbiter tournament and enforcement system.

PART VII — QUALITY & TESTING

Chapter 23: Testing Strategy

Test Runners

- **Vitest** — unit and integration tests. Configured in `vitest.config.ts`.
- **Playwright** — end-to-end tests. Configured in `playwright.config.ts`. Run separately via `pnpm test:e2e`.

Vitest Configuration

```
// vitest.config.ts
export default defineConfig({
  test: {
    environment: 'node',
    globals: true,
    exclude: ['tests/e2e/**', '.next/**', 'node_modules/**'],
    alias: { '@': path.resolve(__dirname, './') },
    maxConcurrency: 5,
    testTimeout: 10000,
  },
});
```

Test Count

36 test files, 251 tests, all passing. Execution time approximately 47 seconds.

Test Types

- **Function tests** — pure function input/output verification (scoring, classification, DTOs)
- **Engine contract tests** — verify engine outputs match contract shapes (`decision-surface.test.ts`, `decision-engine.test.ts`)
- **Component tests** — jsdom environment for React component rendering (`Layout.test.tsx`)
- **Performance benchmarks** — embedded in test files, verify response times
- **Predictive engine tests** — verify deal scoring and routing (`predictive-deal-engine.test.ts`)

Coverage

Coverage is configured for `lib/ai/**/*.ts` with thresholds:

Metric	Threshold
Lines	90%
Functions	95%
Branches	85%
Statements	90%

Chapter 24: Quality Gate

File: `scripts/quality/full-validation.mjs`

The quality gate runs 8 sequential checks. ALL must pass for a CLEAN verdict.

Step	Command	Blocking
1. Prisma validate	`npx prisma validate`	Yes
2. Prisma generate	`npx prisma generate`	Yes
3. TypeScript	`npx tsc --noEmit --pretty false`	Yes
4. PDF audit	`npm run pdf:audit`	Yes
5. MDX integrity	`node scripts/mdx-integrity-check.mjs`	Yes
6. MDX gate	`node scripts/mdx-illegal-jsx-gate.mjs`	Yes
7. Unit tests	`npx vitest run --reporter=verbose`	Yes
8. Build	`npx next build`	Yes

Verdict

- **CLEAN** — all 8 checks pass
- **NOT CLEAN** — any blocking check fails

There is no "clean with debt" verdict. The system previously allowed non-blocking failures but the current implementation treats all checks as blocking.

Running

```
node scripts/quality/full-validation.mjs
# or
pnpm quality:full
```

Chapter 25: Definition of Clean

File: `docs/quality/definition-of-clean.md`

Clean means reproducible under full validation, not "it compiled once."

Requirements

1. `git status` understood — no unexplained modifications or untracked files
2. TypeScript passes with NO exclusions (`pnpm exec tsc --noEmit`)
3. Build passes (`pnpm build`)
4. PDF audit passes (`pnpm pdf:audit`)
5. Unit tests pass (`pnpm test:unit`)
6. No untracked TypeScript files without justification
7. No client/server boundary violations

Boundary Violations

A client/server boundary violation occurs when:

- A file importing `server-only` is imported by a client component
- Scoring logic, thresholds, or classification rules appear in client-bundled code
- Prisma is imported in a client component or edge middleware without dynamic import

---

PART VIII — OPERATIONS & DEPLOYMENT

Chapter 26: Build Pipeline

Build Command

Production builds use `pnpm build`, which runs:

1. `pnpm generate:epubs` — generates EPUB artifacts via `tsx scripts/generate-all-epubs.ts`
2. `next build --webpack` — Next.js production build

The Netlify build uses `pnpm build:netlify`, which runs:

1. `pnpm mdx:gate` — validates MDX JSX patterns
2. `pnpm contentlayer:clean` — cleans stale contentlayer artifacts
3. `pnpm mdx:integrity` — validates MDX component integrity
4. `pnpm build:fast` — builds PDF registry, then runs `next build --webpack` with 7GB memory limit

Pre-commit Hooks

Two checks run as pre-commit hooks:

- **MDX integrity** (`scripts/mdx-integrity-check.mjs`)
- **MDX gate** (`scripts/mdx-illegal-jsx-gate.mjs`)

Prisma Generate

`prisma generate` runs automatically via `postinstall` in `package.json`. It also runs as step 2 of the quality gate.

Webpack Mode

All builds use `--webpack` flag, not Turbopack. This is explicit in every build and dev command.

Chapter 27: Deployment

Platform: Netlify

The application deploys to Netlify using `@netlify/plugin-nextjs`. The `netlify.toml` configuration:

```
[build]
  publish = ".next"
  command = "pnpm build:netlify"

[build.environment]
  NODE_VERSION = "20.20.0"
  NODE_OPTIONS = "--max-old-space-size=7168"
  CI = "true"
  NEXT_TELEMETRY_DISABLED = "1"
  PNPM_VERSION = "10.29.3"
```

Not Vercel

The application does NOT deploy to Vercel. A `vercel.json` file exists in the repository but is not used for production deployment. All deployment configuration targets Netlify.

Required Environment Variables

Variable	Service	Required
`DATABASE_URL`	Neon PostgreSQL	Yes
`NEXTAUTH_SECRET`	NextAuth JWT signing	Yes
`NEXTAUTH_URL`	NextAuth callback URL	Yes
`STRIPE_SECRET_KEY`	Stripe payments	Yes
`STRIPE_WEBHOOK_SECRET`	Stripe webhooks	Yes
`RESEND_API_KEY`	Email delivery	Yes
`UPSTASH_REDIS_REST_URL`	Redis cache	Optional
`UPSTASH_REDIS_REST_TOKEN`	Redis auth	Optional

Netlify Functions

Custom Netlify functions are built from `netlify/functions_src/functions/` using the `nft` bundler. The Prisma client is excluded from function bundles — it resolves from the Netlify runtime layer.

Chapter 28: Environment Management

Database: PostgreSQL Only

The production database is PostgreSQL via Neon. There is no SQLite in the production data path. Legacy SQLite scripts (`db:sqlite:*`) exist in `package.json` for historical reasons but are not part of the active system.

Redis: Optional

Redis via Upstash is optional. The system degrades gracefully:

1. Redis available -> used for rate limiting and caching
2. Redis unavailable -> PostgreSQL fallback for rate limiting
3. Both unavailable -> fail-closed on critical routes, pass-through on non-critical

Graceful Degradation

The `getRedis()` function returns null when Redis is not configured. All Redis consumers check for null and fall through to the next layer. This is not error handling — it is the designed operating mode for environments without Redis.

Chapter 29: Monitoring

SecurityLog

The `SecurityLog` Prisma model records security events: LOGIN_SUCCESS, LOGIN_FAILURE, MFA_CHALLENGE, PASSWORD_CHANGE, UNAUTHORIZED_ACCESS. Each event captures the actor, HTTP method, path, IP address, and user agent.

DiagnosticJourney Snapshots

The `DiagnosticJourney` model stores complete diagnostic lifecycle data. The `MonitoringSnapshot` model captures point-in-time system state for operational observability.

Execution Tracking

- `DecisionJourneyEvent` — records decision lifecycle events with timestamps
- `AuditEvent` — general audit trail for system actions
- `SystemAuditLog` — system-level audit events
- `OperationalIncident` — tracks operational incidents with severity and resolution status
- `ServiceLevelSnapshot` — captures SLA metrics at regular intervals

PART IX — CANONICAL CONTRACT REFERENCE

Chapter 30: Intelligence Spine Contract

File: `lib/decision/intelligence-spine.ts`

The IntelligenceSpine is the single contract object that flows through every diagnostic stage.

```
type IntelligenceSpine = {
  id: string;
  userId?: string;
  email?: string;

  case: CaseObject;
  c3: SpineC3;
  deterministic: DeterministicOutput;
  synthesis: GovernedSynthesis | null;
  forecast: DefaultPathForecast;
  memory: CaseMemoryResult | null;
  kernel: DecisionKernelOutput | null;
  stakeholders: StakeholderMap;

  currentStage: SpineStage;
  history: SpineEvent[];
  createdAt: string;
  updatedAt: string;
};
```

SpineC3

```
type SpineC3 = C3Score & {
  tier: "HARD_RECOVERY" | "SOFT_RECOVERY" | "FULL_SYNTHESIS";
  confidenceBand: "low" | "medium" | "high";
};
```

DeterministicOutput

```
type DeterministicOutput = {
  conditionClass: "authority" | "definition" | "execution" | "instability";
  signal: SignalDefinition;
  contradictionSet: string[];
  blockerClass: string;
};
```

Stage Progression

```
fast_diagnostic -> constitutional -> team -> enterprise ->
executive_reporting -> strategy_room -> outcome_verification
```

Each stage appends a `SpineEvent` to the `history` array. Stages are append-only — regression is detected by `validateIntelligenceSpine()` and flagged as a WARN-level validation error.

Validation

`validateIntelligenceSpine()` checks for:

- Case ID and decision text presence
- C3 score bounds (0-1)
- Tier presence
- HARD_RECOVERY must not have synthesis
- History must be append-only (no stage regression)
- Deterministic condition class must exist
- Timestamps must exist

`assertValidSpine()` throws on BLOCK-level errors. Used at trust boundaries (API routes, DB persistence).

Chapter 31: Constitutional Derivation

The constitutional diagnostic system has a single canonical source in `lib/diagnostics/`. Key files:

- `lib/diagnostics/constitutional-diagnostic-derivation.ts` — canonical derivation logic
- `lib/diagnostics/constitutional-bridge.ts` — bridge between constitutional and fast diagnostic systems
- `lib/diagnostics/constitutional-handoff.ts` — handoff protocol between stages
- `lib/diagnostics/public-constitutional-result.ts` — public DTO definition and sanitizer

The `toPublicResult()` function in `public-constitutional-result.ts` strips all scoring, thresholds, signals, and engine internals from the internal report before it crosses the API boundary.

The constitution version file, if present, is a re-export shim only. The canonical implementation lives in `lib/diagnostics/`.

Chapter 32: Scoring API Contract

POST /api/diagnostics/score

Request:

```
{
  answers: Record<string, string>,
  committed: boolean
}
```

Validated with zod. The `answers` record must contain at minimum a `decision` field with 10+ characters.

Response (success):

Returns `FastDiagnosticResult` (see Chapter 19). The response includes the elevation layer outputs (costOfInaction, executionFailure, authorityIndex, memoryTrend) when the scoring pipeline produces them.

Response (error):

```
{ ok: false, error: string }
```

Error codes: `"Method not allowed"` (405), `"INVALID_REQUEST"` (400), `"Decision text too short"` (400).

Pipeline:

1. Parse and validate request body
2. Create CaseObject from answers
3. Check for contradiction (pre-synthesis)
4. Score C3 fidelity
5. Build deterministic output
6. Forecast default path
7. Synthesize (LLM + arbiter tournament)
8. Compute Cost of Inaction
9. Assess Execution Failure
10. Compute Authority Index
11. Create Decision Memory record
12. Summarize Decision Memory trend
13. Create Intelligence Spine
14. Persist spine to DiagnosticJourney
15. Return sanitized FastDiagnosticResult

PART X — DEVELOPER OPERATIONS

Chapter 33: Local Development Setup

Prerequisites

- Node.js >= 20.0.0
- pnpm >= 9.0.0
- PostgreSQL connection string (Neon or local)

Setup Steps

```
# 1. Clone the repository
git clone <repo-url>
cd aol-check-visual

# 2. Install dependencies
pnpm install

# 3. Configure environment
cp .env.example .env
# Edit .env with your DATABASE_URL, NEXTAUTH_SECRET, etc.

# 4. Generate Prisma client
npx prisma generate

# 5. Push schema to database (or run migrations)
npx prisma db push
# or: npx prisma migrate deploy

# 6. Seed the database (optional)
pnpm db:seed

# 7. Start development server
pnpm dev
```

The dev server runs on `http://localhost:3000` by default.

Environment File

The `.env` file must contain at minimum:

```
DATABASE_URL=postgresql://...
NEXTAUTH_SECRET=<random-secret>
NEXTAUTH_URL=http://localhost:3000
```

Optional but recommended for full functionality:

```
UPSTASH_REDIS_REST_URL=...
UPSTASH_REDIS_REST_TOKEN=...
STRIPE_SECRET_KEY=...
STRIPE_WEBHOOK_SECRET=...
RESEND_API_KEY=...
```

Chapter 34: Development Workflow

Branch Strategy

Branch from `main`. The main branch is the production branch.

Before Pull Request

Run the full quality gate:

```
node scripts/quality/full-validation.mjs
```

All 8 checks must pass. Do not submit a PR with failing checks.

Commit Discipline

- No force push to main
- Commit messages should describe intent, not just changes
- Keep commits atomic — one logical change per commit

TypeScript

TypeScript must compile with zero errors and no exclusions:

```
npx tsc --noEmit
```

The `tsconfig.json` uses strict mode with additional strictness flags (`noUncheckedIndexedAccess` , `noImplicitOverride` , `noFallthroughCasesInSwitch`). Target is ES2022 with bundler module resolution.

Server-Only Boundary

When adding new scoring, classification, or synthesis logic:

1. Place the file in `lib/server/` or `lib/decision/`
2. Add `import "server-only"` at the top
3. Export only public-safe types and functions
4. Never import server-only modules from Pages Router page components — use API routes instead

Chapter 35: Scripts Reference

Quality & Testing

Script	Command	Purpose
`quality:full`	`node scripts/quality/full-validation.mjs`	Full validation gate (8 checks)
`test:unit`	`vitest run`	Run unit + integration tests
`test:e2e`	Playwright	Run end-to-end tests
`pdf:audit`	`npm run pdf:audit`	Audit PDF artifacts

Build & Development

Script	Command	Purpose
`dev`	`pnpm mdx:gate && ... && next dev --webpack`	Start dev server
`build`	`pnpm generate:epubs && next build --webpack`	Production build
`build:netlify`	MDX gate + contentlayer clean + integrity + fast build	Netlify-specific build
`lint`	`tsc --noEmit`	TypeScript check (ESLint has broken ajv dep)

Database

Script	Command	Purpose
`db:generate`	`prisma generate`	Generate Prisma client
`db:push`	`prisma db push`	Push schema without migration
`db:migrate`	`prisma migrate dev`	Create new migration
`db:deploy`	`prisma migrate deploy`	Apply migrations (production)
`db:seed`	`tsx prisma/seed.ts`	Seed database
`db:studio`	`prisma studio`	Open Prisma Studio GUI
`db:reset`	`prisma migrate reset --force && pnpm db:seed`	Reset and reseed

Content

Script	Command	Purpose
`mdx:gate`	`node scripts/mdx-illegal-jsx-gate.mjs`	Block illegal JSX in MDX
`mdx:integrity`	`node scripts/mdx-integrity-check.mjs`	Validate MDX component tags
`content:validate`	`node scripts/validate-frontmatter.mjs`	Validate content frontmatter
`contentlayer:build`	`contentlayer2 build`	Build content layer

Chapter 36: Troubleshooting

Server-Only Import in Pages Router

Problem: Build fails with "server-only" import error in a Pages Router page.

Cause: A page component (rendered on both server and client) imports a module that uses `import "server-only"`. The Pages Router does not support the `server-only` package in page components.

Fix: Move the server-only call to an API route. The page should fetch data from `/api/...` instead of importing the server module directly. Only App Router Server Components can import `server-only` modules directly.

ioredis Client Bundling

Problem: Build fails or bundle size explodes with ioredis in the client bundle.

Cause: A client component or shared module imports Redis utilities.

Fix: Redis imports must be dynamic (`await import(...)`) or isolated in server-only files. The `getRedis()` function should only be called from API routes or server components. Use `lib/server/security/persistent-rate-limit.ts` as the entry point — it handles the Redis/Postgres fallback internally.

Dual ESLint Configs

Problem: ESLint produces inconsistent results or fails with ajv dependency errors.

Cause: The project has `eslint.config.mjs` (flat config format). The `ajv` dependency used by some ESLint plugins has known compatibility issues.

Fix: Use `tsc --noEmit` as the primary lint check. The quality gate uses TypeScript compilation rather than ESLint. When ESLint is needed, run it separately and treat `ajv`-related errors as non-blocking.

Contentlayer Windows Issues

Problem: Contentlayer2 fails on Windows with path resolution errors.

Fix: Use `cross-env IS_WINDOWS=true` or run `pnpm dev:windows`. The `contentlayer.config.ts` detects Windows via `process.platform === "win32"` and adjusts path handling. If issues persist, run `pnpm contentlayer:safe` which clears the cache first.

Prisma Client Not Generated

Problem: TypeScript errors about missing Prisma types.

Fix: Run `npx prisma generate`. This runs automatically on `pnpm install` via the `postinstall` script, but may need to be run manually after schema changes or fresh checkouts.

APPENDICES

Appendix A: Prisma Model Inventory

126 models in `prisma/schema.prisma`:

Model	Domain
Organisation	Enterprise
AlignmentCampaign	Enterprise
AlignmentSnapshot	Enterprise
OrganisationInvite	Enterprise
OrganisationMembership	Enterprise
GovernanceLog	Governance
User	Identity
Account	Identity
VerificationToken	Identity
Entitlement	Access
AccessKey	Access
AccessKeyUse	Access
AccessAuditLog	Audit
AccessInvite	Access
CampaignParticipant	Enterprise
AuditResponse	Enterprise
EnterpriseAssessment	Enterprise
TeamAssessmentSnapshot	Enterprise
TeamAssessmentCampaign	Enterprise
TeamAssessmentInvite	Enterprise
TeamAssessmentResponse	Enterprise
TeamAssessmentAggregate	Enterprise
OrganisationAssessmentSnapshot	Enterprise
LeadershipGapSnapshot	Enterprise
EnterpriseReport	Enterprise
DiagnosticRecord	Diagnostics
DiagnosticReportOrder	Diagnostics
DiagnosticArtifact	Diagnostics

Model	Domain
DiagnosticRegenerationJob	Diagnostics
DiagnosticAuditEvent	Diagnostics
DiagnosticArtifactAccessGrant	Diagnostics
DiagnosticLineageEvent	Diagnostics
ProofEvidence	Diagnostics
ClientEntitlement	Access
BillingCustomer	Commerce
OperationalIncident	Operations
ArtifactManifest	Operations
JobDeadLetter	Operations
ServiceLevelSnapshot	Operations
RunbookEntry	Operations
DecisionRecommendationSession	Decision
DecisionRecommendationImpression	Decision
DecisionRecommendationClick	Decision
DecisionRecommendationConversion	Decision
DecisionSessionFollowup	Decision
DecisionAssetEfficacy	Decision
DecisionAssetContextPerformance	Decision
GovernanceMetricDefinition	Governance
DecisionSignalRegistry	Decision
DecisionGovernanceAlert	Decision
PurposeAlignmentAssessment	Alignment
PurposeAlignmentReport	Alignment
PurposeAlignmentReminderPreference	Alignment
PurposeAlignmentReminderLog	Alignment
DealFlowSubmission	Commerce
DecisionAssetPerformance	Decision

Model	Domain
InnerCircleMember	Access
AdminSession	Identity
ApiKey	Identity
ApiLog	Operations
InnerCircleKey	Access
MfaSetup	Identity
Session	Identity
SecurityLog	Security
PageView	Analytics
RateLimitLog	Security
SystemAuditLog	Audit
UserPreference	Identity
UserIntegration	Identity
StrategyInquiry	Strategy
StrategyIntake	Strategy
ConstitutionalIntakeReport	Diagnostics
ExecutiveReportingRun	Enterprise
ExecutiveReportingArtifact	Enterprise
DiagnosticJourney	Diagnostics
DiagnosticEvidenceNode	Diagnostics
DiagnosticDecisionObject	Diagnostics
DecisionDependency	Decision
DecisionStakeholder	Decision
StakeholderPosition	Decision
AuditEvent	Audit
EnforcementPlaybook	Governance
PlaybookApplication	Governance
FoundationTelemetryEvent	Operations

Model	Domain
RetainerContract	Commerce
RetainedDecision	Commerce
EnforcementCycle	Governance
DiagnosticStageRecord	Diagnostics
LongitudinalComparisonRecord	Diagnostics
MultiStakeholderResult	Diagnostics
OutcomeVerificationRecord	Diagnostics
DiagnosticThreadSnapshot	Diagnostics
MonitoringSnapshot	Operations
BenchmarkFact	Operations
BenchmarkCohortSnapshot	Operations
StrategyRoomSession	Strategy
StrategyRoomRecommendationImpression	Strategy
StrategyRoomFollowup	Strategy
StrategyRoomConversion	Strategy
AdminDecisionContextualEfficacy	Decision
ContentMetadata	Content
Framework	Content
StrategicLink	Content
ContentRelation	Content
PrivateAnnotation	Content
CanonEntry	Content
StrategicFramework	Content
StrategicIntervention	Strategy
CorrectionNode	Governance
DecisionAssetGovernanceRule	Decision
DownloadAuditEvent	Commerce
PrintAsset	Content

Model	Domain
PremiumDownloadToken	Commerce
PremiumDownloadAttempt	Commerce
FrameworkAccessLog	Content
DecisionJourneyEvent	Decision
StrategyRoomExecutionSession	Strategy
StrategyDecisionLog	Strategy
FailedEntitlementGrant	Access
ConsequenceTimeline	Decision
EscalationEvent	Governance
CalibrationState	Operations
CalibrationEvent	Operations
PatternBreakerContract	Governance
RateLimitBucket	Security
DecisionMemory	Decision

Appendix B: API Route Inventory

All routes under `pages/api/`:

Route	Method	Auth Required
`/api/diagnostics/score`	POST	No
`/api/diagnostics/submit`	POST	No (rate-limited)
`/api/diagnostics/[ref]`	GET	Optional
`/api/diagnostics/list`	GET	Yes
`/api/diagnostics/report`	GET	Yes
`/api/diagnostics/create-report-checkout`	POST	Yes
`/api/diagnostics/constitutional-intake/*`	POST/GET	Varies
`/api/diagnostics/enterprise`	POST	Yes
`/api/diagnostics/executive-reporting`	POST	Yes
`/api/diagnostics/team-alignment`	POST	Yes
`/api/diagnostics/spine/*`	GET	Yes
`/api/auth/[...nextauth]`	GET/POST	No
`/api/auth/identity`	GET	No
`/api/auth/me`	GET	Yes
`/api/auth/session`	GET	No
`/api/auth/mint`	POST	Yes
`/api/auth/sovereign-login`	POST	Dev only
`/api/stripe/*`	POST	Webhook secret
`/api/health`	GET	No
`/api/contact`	POST	No (rate-limited)
`/api/subscribe`	POST	No (rate-limited)
`/api/downloads/*`	GET	Varies
`/api/inner-circle/*`	GET/POST	Yes (inner_circle+)
`/api/admin/*`	GET/POST/PUT/DELETE	Yes (admin+)
`/api/strategy-room/*`	GET/POST	Yes
`/api/ogr/*`	GET/POST	Yes
`/api/billing/*`	GET/POST	Yes

Appendix C: Environment Variable Reference

Variable	Required	Description
`DATABASE_URL`	Yes	PostgreSQL connection string (Neon)
`NEXTAUTH_SECRET`	Yes	JWT signing secret
`NEXTAUTH_URL`	Yes	Application base URL
`STRIPE_SECRET_KEY`	Yes	Stripe API secret key
`STRIPE_PUBLISHABLE_KEY`	Yes	Stripe publishable key
`STRIPE_WEBHOOK_SECRET`	Yes	Stripe webhook signing secret
`RESEND_API_KEY`	Yes	Resend email API key
`UPSTASH_REDIS_REST_URL`	No	Upstash Redis REST endpoint
`UPSTASH_REDIS_REST_TOKEN`	No	Upstash Redis auth token
`RECAPTCHA_SECRET_KEY`	No	reCAPTCHA server secret
`RECAPTCHA_SITE_KEY`	No	reCAPTCHA client key
`NODE_ENV`	Auto	`development` or `production`
`CI`	Auto	Set to `true` in CI environments

Appendix D: Quality Gate Checklist

Before any merge to main, verify:

- ☐ `git status` — no unexplained untracked or modified files
- ☐ `npx prisma validate` — schema is valid
- ☐ `npx prisma generate` — client is generated
- ☐ `npx tsc --noEmit` — zero TypeScript errors, no exclusions
- ☐ `pnpm pdf:audit` — PDF audit passes
- ☐ `node scripts/mdx-integrity-check.mjs` — MDX integrity passes
- ☐ `node scripts/mdx-illegal-jsx-gate.mjs` — MDX gate passes
- ☐ `npx vitest run` — all 251 tests pass
- ☐ `next build --webpack` — build succeeds
- ☐ No `server-only` imports in client-bundled code
- ☐ No scoring logic, thresholds, or classification rules in client bundle
- ☐ No secrets in committed files

Or run all at once:

```
node scripts/quality/full-validation.mjs
```

End of Engineering Manual v2.0